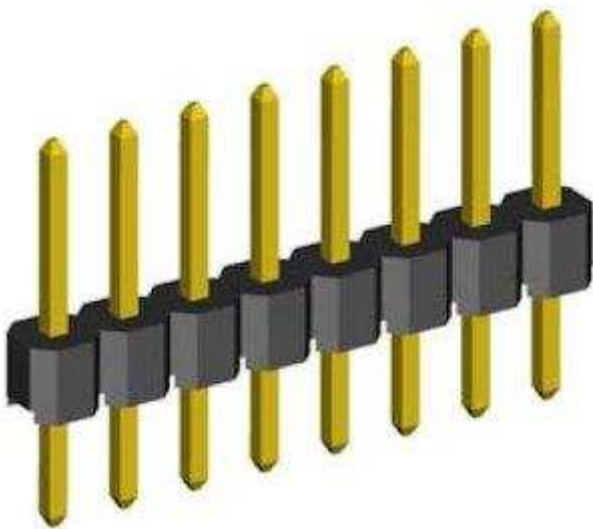


BASIC CONCEPTS OF COMPUTER SCIENCE

ALGORITHMS, PROGRAMING LANGUAGES, AND
USER INTERFACES



Abstract

The concept of an algorithm, the concept of a programming language, and the concept of a user interface

By

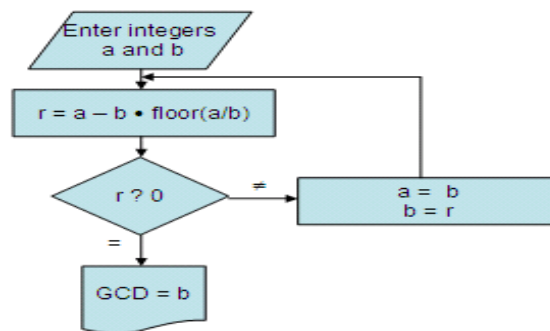
Caleb McIvor

Contents

ALGORITHMS:	2
Programing Languages:	5
User Interface:	7
Bibliography.....	10

ALGORITHMS:

Basic concepts of computer science are: the concept of an algorithm, the concept of a programming language, and the concept of a user interface and its usability. The algorithm I am studying is linear search, which is used to search for a number in a list.



An algorithm is a precise unambiguous specification of how to accomplish a computational task in a finite number of well-defined steps. An algorithm is distinct from a computer program, and is often expressed in pseudo code or a flow chart. An algorithm has a cost (the number of steps it will take to complete a task). Different algorithms have different costs for the same task some being better than others.

```
public MostRecentFile[] GetFileList()
{
    if (!Loaded)
    {
        LoadMruList();
    }
    object[] array = files.ToArray();
    MostRecentFile[] mrfArray = new MostRecentFile[array.Length];
    array.CopyTo(mrfArray, 0);
    return mrfArray;
}

public bool Contains(string fileName)
{
    if (!Loaded)
    {
        LoadMruList();
    }
}
```

A programming language is a precise, formal language for writing programs that can be run on a computer; it is distinct from pseudo code and natural language. There are different levels of programming languages; programs can be translated from high level to low level (machine) languages. A programming language could be a drag-and-drop language such as scratch, or a specialised programming language, or a general-purpose language.

An informal instruction is a brief imprecise command for example go to number 5 instead of check number 1 if = 5 print done. An informal instruction could be risky because the computer may not understand the command.

Algorithms are distinct than programming languages because algorithms are a set of instructions and a programming language is those instructions but in a formal language that a computer can understand. Algorithms are different than informal languages because algorithms are precise instructions that have a step-by-step process that has a beginning and an end and informal languages are direct instructions that are brief and non-descriptive.

An algorithm is a set of instructions which to be used on a computer it needs to be in a formal language so the computer understands the instructions. So an algorithm is the plan of what to do whereas the program is the thing that will perform those instructions.

An example of an algorithm is an activity with a list of numbers as an example I will use: 1, 4, 9, 14, 16, 23, 32 now if I needed to find number 23 with a linear search I would:

- Take the first number in this case '1' and see if it is equal to the number I need '23' $1 = 23$ which is false '1' does not equal '23'.
- So I then go to the second number '4' and see if it is equal to '23' $4 = 23$ which is false '4' does not equal '23'.
- Next I go to the third number '9' and see if it is equal to '23' $9 = 23$ which is false '9' does not equal '23'.
- So I then go to the fourth number '14' and see if it is equal to '23' $14 = 23$ which is false '14' does not equal '23'.
- So I then go to the fifth number '16' and see if it is equal to '23' $16 = 23$ which is false 16 does not equal 23.
- So I then go to the sixth number '23' and see if it is equal to '23' $23 = 23$ which is true so the number was found in 6 steps.

But if you compare this to binary searching with the same list:

1, 4, 9, 14, 16, 23, 32 the efficiency will be different. One thing with binary searching is the list has to be in order. To start with the binary search you:

- Take the middle number which is '14' and check to see if the number you are looking for is less than that number so looking for '23' again is '23' less than '14' $23 < 14$ no it is not so everything less than 14 is ignored: (~~1, 4, 9~~, 14, 16, 23, 32).
- Next you take the number in the middle of the last middle number and the end which is '23' so is '23' less than '23' $23 < 23$ no so everything less than '23' is ignored: (~~1, 4, 9, 14, 16~~, 23, 32)
- Then take the middle number between the last middle number and the end which is '32' and see if '23' is less than '32' $23 < 32$ which it is so everything greater than '32' is ignored: (~~1, 4, 9, 14, 16~~, 23, ~~32~~)
- this just leaves 23 so then I check if the number remaining is equal to the number I'm trying to find $23 = 23$ which it is so the number is found in 4 steps.

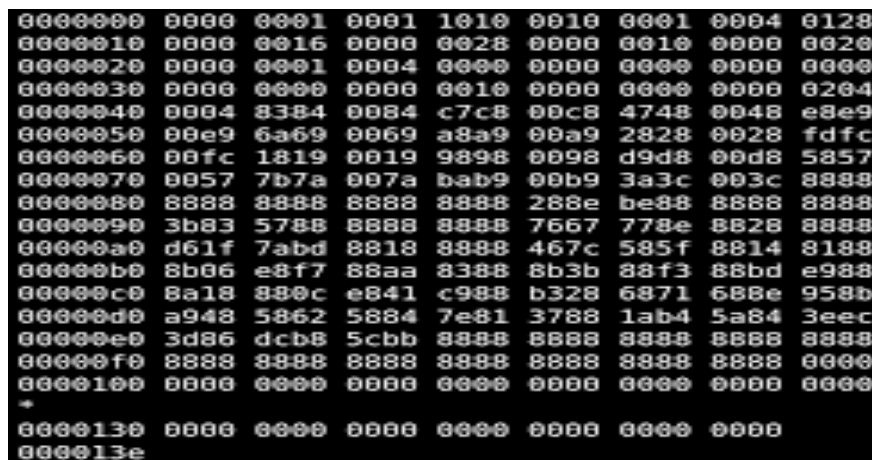
In conclusion binary searching is more efficient because it takes fewer steps to find the number in this case 2 less. This is because linear searching has to check each and every number until the number is found. So at best if the number is at the start of the list the cost will be 1 at best but at worse the cost could be n (the total number of numbers in the list) whereas in a binary search it looks at the middle number and knows the desired number can only be on one side so ignores the rest. The cost of a binary search is $\log_2(N)+1$, which is better in most cases.

This is a table comparing the cost of linear and binary searches.

Array Size	Linear – N	Binary – $\log_2 N$
10	10	4
50	50	6
100	100	7
500	500	9
1000	1000	10
2000	2000	11
3000	3000	12
4000	4000	12
5000	5000	13
6000	6000	13
7000	7000	13
8000	8000	13
9000	9000	14
10000	10000	14

Programing Languages:

A programing language is a precise, formal language for writing programs that can be run on a computer, it is distinct from pseudo code and natural language. There are different levels of programming languages; programs can be translated from high level to low level (machine) languages. A programming language could be a drag-and-drop language such as scratch, or a specialised programing language, or a general-purpose language.



A role for a programming language might be to calculate a number with any given algorithm and with a low level language this would be difficult to write but will run faster. In a low level programing language it would be a lot easier to write but have speed disadvantages. The reason a high level language is slower is because the processor does not directly understand it. The program needs to be translated to a low level language so the computer can understand it this is called compiling. So the low level language can directly run on the processor but the high level language needs to be interpreted which takes time therefore slowing the program down. So if speed is essential for the application a low level language is the one to use but if not a high level language would be much easier. A high level programing language is a programing language that is not far off or is machine code. A high level programing language is quite far from machine code and is easier to learn.

This is an example of a liner search program in c which is a high level programing language:

```
#include<stdio.h>
int main() {

    int a[10],i,n,m,c=0;

    printf("Enter the size of an array: ");
```

```

scanf("%d",&n);

printf("Enter the elements of the array: ");
for(i=0;i<=n-1;i++){
    scanf("%d",&a[i]);
}

printf("Enter the number to be search: ");
scanf("%d",&m);
for(i=0;i<=n-1;i++){
    if(a[i]==m){
        c=1;
        break;
    }
}
if(c==0)
    printf("The number is not in the list");
else
    printf("The number is found");

return 0;
}

```

Sample output:

```

Enter the size of an array: 5
Enter the elements of the array: 4 6 8 0 3
Enter the number to be search: 0
The number is found

```

```

def add5(x):
    return x+5

def dotwrite(ast):
    nodename = getNodeName()
    label=symbol.sym_name.get(int(ast[0]),ast[0])
    print " %s [label=\"%s\" % (nodename, label),
    if isinstance(ast[1], str):
        if ast[1].strip():
            print " %s";" % ast[1]
        else:
            print ""
    else:
        print "[]";
        children = []
        for n, child in enumerate(ast[1:]):
            children.append(dotwrite(child))
        print " %s -> {" % nodename,
        for name in children:
            print " %s" % name,

```

A high level language is easy for a human to understand but hard for a computer to understand and a low level language is easy for a computer to understand but hard for a human to understand.

When you make a program in a high level language you can compile (translate) it to a low-level language program. The advantage of this is a speed increase in the program. You can also translate a low level language to a high level language. The advantage of this is the new program will be easier to understand and edit.

Some similarities between high and low level programming languages are that they're both programming languages and can run the same algorithms. A high level language can do everything a low level language can do. The differences are the speed they take to complete the task and how easy they are to understand.

User Interface:

A user interface is the part of the computer or electronic system that a human user interacts with to control the system. The usability of an interface is the key characteristic for evaluating the interface.

In a user interface different factors contribute to its useability for example:

- The system should always keep users informed about what is going on through appropriate feedback within reasonable time
- The system should speak the users language
- Users often choose system functions by mistake and will need a clearly marked "emergency exit" to leave the unwanted state without having to go through an extended dialogue. Supports undo and redo.
- Users should not have to wonder whether different words, situations, or actions mean the same thing. Follow platform conventions.
- Even better than good error messages is a careful design, which prevents a problem from occurring in the first place. Either eliminates error-prone conditions or check for them and present users with a confirmation option before they commit to the action.
- Minimize the user's memory load by making objects, actions, and options visible. The user should not have to remember information from one part of the dialogue to another. Instructions for use of the system should be visible or easily retrievable whenever appropriate.
- Accelerators -- unseen by the novice user -- may often speed up the interaction for the expert user such that the system can cater to both inexperienced and experienced users. Allow users to tailor frequent actions.
- Dialogues should not contain information, which is irrelevant or rarely needed. Every extra unit of information in a dialogue competes with the relevant units of information and diminishes their relative visibility.
- Error messages should be expressed in plain language (no codes), precisely indicate the problem, and constructively suggest a solution.
- Even though it is better if the system can be used without documentation, it may be necessary to provide help and documentation. Any such information should be easy to search, focused on the user's task, list concrete steps to be carried out, and not be too large.

These different factors contribute to user interfaces usability. To show this I will relate two different user interfaces that perform a similar task. The two I will use are:

- IOS
- Android



<u>IOS</u>	<u>ANDRIOD</u>
Home screen: You can rearrange app shortcuts and place them in folders. You cannot have live wallpapers or widgets on the home screen.	Home screen: You can rearrange app shortcuts and place them in folders. You can have custom live wallpapers and widgets on the home screen.
Apps: All apps are placed on the home screen and stay there. There is no way to remove them off the home screen without deleting the app. There is no other menu to access all of the apps.	Apps: All of the apps can be found in the “all applications” menu and shortcuts to them can be made on the home screen. Some apps also have widgets that you can place on the home screen instead of a shortcut.
File Management: IOS offers no kind of file management so there is no way to see all of your files or organise them.	File Management: Android has a file management system where you can access all files on the phone or memory card. All of these files you can edit delete and move.
Languages: IOS supports 34 different languages	Languages: Android supports 40 different languages
Camera: There is a shortcut to the camera on the lock screen	Camera: There is a shortcut to the camera on the lock screen.
Status Bar: IOS has a status bar that displays battery percentage, Wi-Fi strength, cell signal, and other app notifications.	Status Bar: Android has a status bar that displays battery percentage, Wi-Fi strength, cell signal, and other app notifications.

Going just off this information I would choose an Android based system as it has some features that the IOS does not but IOS does not have any features that Android does not have.

Bibliography

<http://www.matrixlab-examples.com/image-files/euclidean-algorithm-control-flow.gif>

http://upload.wikimedia.org/wikipedia/commons/1/17/C_Sharp_Code.PNG

<http://www.host-shopper.com/assets/images/sample-python-code.png>

http://upload.wikimedia.org/wikipedia/commons/thumb/7/76/Wikipedia_favicon_hexdump.svg/1188px-Wikipedia_favicon_hexdump.svg.png

<http://www.nngroup.com/articles/ten-usability-heuristics/>

<http://en.wikipedia.org/wiki/IOS>

- http://upload.wikimedia.org/wikipedia/en/7/7d/IOS_6_Home_Screen.png

- https://lh3.ggpht.com/0m1qq1VRBqh8ZD8ryFFBEu-EXZmHrOfa5JJMjZhRQA-aOgdjTEa_2DP8se6qrs-6_Nc=h900

<http://www.csit.parkland.edu/~mbrandyberry/CS1Java/images/Lesson27/SearchOrderComparisonTable.jpg>

<http://www.cquestions.com/2008/01/c-program-for-linear-search.html>